

Sheet2_Neural network

March 10, 2023

Name	ID	Department
Abdelrhman Mohamed Abdelhady Hodib	2022513643	Intelligent system

1 Questions

1.1 How can you create a DataFrame in Pandas from a Numpy array or a Python list?

you can use the `pandas.DataFrame()` function and pass the NumPy array as an argument.

```
[1]: import pandas as pd
import numpy as np

data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
df = pd.DataFrame(data, columns=['A', 'B', 'C'])
```

1.2 How can you perform operations on DataFrame columns and rows in Pandas?

To perform operations on DataFrame columns and rows in Pandas, you can use methods like `.apply()`, `.map()`, `.applymap()`, and indexing. For example, to add a constant value to a column, you can use:

```
[2]: df['A'] = df['A'] + 5
```

To apply a function to each element of a column, you can use `.apply()`:

```
[3]: df['A'] = df['A'].apply(lambda x: x ** 2)
```

To select rows based on a condition, you can use indexing:

```
[4]: df[df['A'] > 5]
```

```
[4]:   A  B  C
0  36  2  3
1  81  5  6
2 144  8  9
```

1.3 How can you sort a DataFrame in Pandas?

To sort a DataFrame in Pandas, you can use the `.sort_values()` method. For example, to sort a DataFrame by a single column:

```
[5]: df.sort_values('A', ascending=False)
```

```
[5]:      A  B  C
2  144  8  9
1   81  5  6
0   36  2  3
```

To sort by multiple columns:

```
[6]: df.sort_values(['A', 'B'], ascending=[False, True])
```

```
[6]:      A  B  C
2  144  8  9
1   81  5  6
0   36  2  3
```

1.4 How can you drop missing values from a DataFrame in Pandas?

To drop missing values from a DataFrame in Pandas, you can use the `.dropna()` method. For example, to drop rows with any missing values:

```
[7]: df.dropna()
```

```
[7]:      A  B  C
0   36  2  3
1   81  5  6
2  144  8  9
```

To drop columns with any missing values:

```
[8]: df.dropna(axis=1)
```

```
[8]:      A  B  C
0   36  2  3
1   81  5  6
2  144  8  9
```

1.5 How can you perform groupby operations on a DataFrame in Pandas?

To perform groupby operations on a DataFrame in Pandas, you can use the `.groupby()` method. For example, to group by a single column and calculate the mean of another column:

```
[9]: df.groupby('A')['B'].mean()
```

```
[9]: A
     36      2.0
     81      5.0
    144      8.0
     Name: B, dtype: float64
```

To group by multiple columns and calculate the sum of another column:

```
[10]: df.groupby(['A', 'B'])['C'].sum()
```

```
[10]: A    B
     36  2    3
     81  5    6
    144  8    9
     Name: C, dtype: int32
```

1.6 How can you merge two DataFrames in Pandas based on a common column or index?

To merge two DataFrames in Pandas based on a common column or index, you can use the `.merge()` method. For example, to merge two DataFrames based on a common column:

```
[11]: data = [{'A': 1, 'B': 2}, {'A': 3, 'B': 4, 'C': 5}]
      df1 = pd.DataFrame(data)
      data = [{'A': 1, 'B': 2}, {'A': 3, 'B': 4, 'C': 5}]
      df2 = pd.DataFrame(data)

      df1.merge(df2, on='A')
```

```
[11]:   A  B_x  C_x  B_y  C_y
0  1    2  NaN    2  NaN
1  3    4  5.0    4  5.0
```

To merge based on a common index:

```
[12]: df1.merge(df2, left_index=True, right_index=True)
```

```
[12]:   A_x  B_x  C_x  A_y  B_y  C_y
0    1    2  NaN    1    2  NaN
1    3    4  5.0    3    4  5.0
```

1.7 How can you calculate the mean, median, and mode of a Pandas DataFrame column?

To calculate the mean, median, and mode of a Pandas DataFrame column, you can use the `.mean()`, `.median()`, and `.mode()` methods, respectively. For example:

```
[13]: df['A'].mean()
      df['A'].median()
```

```
df['A'].mode()
```

```
[13]: 0      36
      1      81
      2     144
      Name: A, dtype: int64
```

1.8 What is the difference between iloc and loc in Pandas?

The difference between `.iloc` and `.loc` in Pandas is in how they access rows and columns. `.iloc` uses integer indices to select rows and columns, while `.loc` uses labels. For example, to select the first row and second column:

```
[14]: print(df.iloc[0, :])
      print(df.iloc[:, 1])
```

```
A      36
B       2
C       3
Name: 0, dtype: int64
0       2
1       5
2       8
Name: B, dtype: int32
```

```
[15]: #Access column B
      print(df.loc[:, 'B'])
```

```
0       2
1       5
2       8
Name: B, dtype: int32
```

1.9 Coding Questions

1.10 1st Question

1.10.1 Generate a pandas data frame contains 10 entries with the following columns: Name[A - Z], Age [7 - 90], Gender [M / F], and City [A - Z] and fill it with randomly generated Numbers or Alphabetical Characters as shown with brackets.

```
[16]: data = {'Name': np.random.choice(list('ABCDEFGHIJKLMNOPQRSTUVWXYZ'), 10),
              'Age': np.random.randint(7, 91, 10),
              'Gender': np.random.choice(['M', 'F'], 10),
              'City': np.random.choice(list('ABCDEFGHIJKLMNOPQRSTUVWXYZ'), 10)}

df = pd.DataFrame(data)
print(df)
```

	Name	Age	Gender	City
0	T	16	F	A
1	Y	48	M	T
2	K	14	M	Z
3	R	24	M	D
4	H	27	F	R
5	V	81	F	T
6	I	26	M	J
7	L	84	M	C
8	D	24	F	D
9	V	41	M	Q

1.10.2 Create a new column called `Category` that assigns each person to a category based on their age: "Child" for ages under 18, "Adult" for ages 18-65, and "Senior" for ages over 65. Then, count the number of people in each category and display the result.

```
[17]: df['Category'] = pd.cut(df['Age'], bins=[0, 18, 65, np.inf], labels=['Child',
    ↪ 'Adult', 'Senior'])
```

```
[18]: category_counts = df['Category'].value_counts()
print(category_counts)
```

```
Adult      6
Child      2
Senior     2
Name: Category, dtype: int64
```

1.11 2nd Question

1.11.1 Generate a pandas data frame that contains 10 rows of information about different movies, including the title [A - Z], genre["Comedy" / "Drama" / "Action"], year[1990 - 2010], and rating [1 - 10].

```
[19]: np.random.seed(1)
df = pd.DataFrame({'Title': np.random.
    ↪ choice(list('ABCDEFGHIJKLMNOPQRSTUVWXYZ'), size=10),
    'Genre': np.random.choice(['Comedy', 'Drama', 'Action'],
    ↪ size=10),
    'Year': np.random.randint(1990, 2011, size=10),
    'Rating': np.random.uniform(1, 10, size=10)})
print(df)
```

	Title	Genre	Year	Rating
0	F	Drama	2008	8.005503
1	L	Comedy	2010	7.443735
2	M	Drama	2001	8.224818
3	I	Comedy	2000	1.835207
4	J	Action	2004	5.663373

5	L	Drama	2008	8.785182
6	F	Action	1994	8.462322
7	P	Comedy	1999	8.466430
8	A	Action	2007	3.457450
9	Q	Drama	1990	1.533189

1.11.2 Create a new data frame that contains only the movies from the year 2000 or later and with a rating of 7.0 or higher. Then, sort the data frame by rating in descending order and display the top 10 movies.

```
[20]: df_filtered = df[(df['Year'] >= 2000) & (df['Rating'] >= 7.0)]
print(df_filtered, "\n")

df_top10 = df_filtered.sort_values(by='Rating', ascending=False).head(10)
print(df_top10)
```

	Title	Genre	Year	Rating
0	F	Drama	2008	8.005503
1	L	Comedy	2010	7.443735
2	M	Drama	2001	8.224818
5	L	Drama	2008	8.785182

	Title	Genre	Year	Rating
5	L	Drama	2008	8.785182
2	M	Drama	2001	8.224818
0	F	Drama	2008	8.005503
1	L	Comedy	2010	7.443735